RHEL / DevOps

Scenario-Based Interview & Revision Prep

15 Topics | 60 Scenario Questions with Answers & Commands

Contents

1. Deployment

Q1. Your team needs to provision a new RHEL 9 application server from scratch and configure it consistently with 20 other servers. How would you approach this instead of doing it manually?

Answer:

I'd avoid manual, one-off installs and use an Infrastructure-as-Code approach. In my homelab I use an Ansible control node (RHEL 9) with an inventory of managed hosts, connecting over SSH key-based auth. I'd write idempotent playbooks/roles that handle base OS configuration (users, packages, firewall, SELinux context, services) and run them with ansible-navigator against an execution environment image so the tooling is consistent across runs. This guarantees the new server matches the other 20 exactly, and any drift can be corrected by simply re-running the playbook.

Q2. You've built a Kickstart or Agent-based install for OpenShift/RHEL, but the deployment fails midway on one node in a batch of ten. How do you troubleshoot without reinstalling all ten?

Answer:

First isolate the failing node rather than assuming it's systemic. I check the console/serial log or, for an Agent-based OpenShift install, the assisted-installer logs and 'openshift-install agent wait-for bootstrap-complete --log-level debug'. Common root causes are DNS/DHCP mismatches, NTP/time skew, insufficient CPU/RAM against the node's role, or a bad network config in the install-config.yaml/agent-config.yaml. I fix the specific node's config (e.g., correct MAC-to-IP mapping) and re-trigger only that node's install, since Kickstart/Agent-based installs are per-node and don't require touching the healthy nodes.

Q3. A junior engineer asks: what's the practical difference between deploying a single VM manually through virt-manager/ESXi vs. deploying it through an automated pipeline?

Answer:

Manual deployment through ESXi/virt-manager is fine for a one-off lab VM, but it isn't repeatable, isn't documented as code, and is error-prone under time pressure. An automated pipeline (Ansible, Kickstart, or CI/CD triggering Terraform+Ansible) captures every step as version-controlled code, so the same result can be reproduced on demand, rolled back, and peer-reviewed. In my own homelab I still deploy the base ESXi VM shell manually, but everything past that — OS config, packages, hardening — is done through Ansible so it's consistent and auditable.

Q4. How would you validate a deployment before declaring it 'production-ready'?

Answer:

I run a validation checklist: service health (systemctl status / is-active), port reachability, SELinux denials in /var/log/audit/audit.log, firewalld rules matching the intended ports, DNS resolution both forward and reverse, and a smoke test hitting the actual application endpoint. For a clustered deployment like SNO/OpenShift, I'd additionally check node readiness with 'oc get nodes' and cluster operators with 'oc get co' to confirm everything reports Available=True before sign-off.

2. User & Group Management

Q1. A new DevOps engineer joins your team and needs a login account that can run sudo only for specific commands, not full root. Walk me through it.

Answer:

I'd create the user with useradd, set a password with passwd (or force a reset on first login with chage -d 0), then instead of adding them to the wide-open 'wheel' group I'd grant limited privileges through a dedicated sudoers drop-in file in /etc/sudoers.d/. That file specifies only the exact commands they need (e.g. systemctl restart httpd, journalctl) using visudo to edit safely so syntax errors don't lock out sudo entirely.

```
useradd -m -c "New Engineer" jdoe
passwd jdoe
chage -d 0 jdoe
visudo -f /etc/sudoers.d/jdoe
# jdoe ALL=(root) NOPASSWD: /bin/systemctl restart httpd, /bin/journalctl
```

Q2. You find that a shared group (say 'devops') needs its members to collaborate on files in /data/projects, but new files created by different users keep landing with the wrong group ownership. How do you fix it structurally?

Answer:

This is a classic case for the SGID bit on the directory rather than fixing files after the fact. I'd set group ownership of /data/projects to devops, then set the setgid bit so any new file or subdirectory created inside automatically inherits the devops group instead of the creator's primary group. I'd also set a sane default umask (or an ACL default) so group-write permissions are consistent for every new file.

```
groupadd devops
chgrp -R devops /data/projects
chmod 2775 /data/projects
# 2 = setgid bit -> new files inherit 'devops' group
```

Q3. How do you audit which users belong to a critical group like 'wheel' and remove someone who left the company, without breaking their existing files?

Answer:

I list membership with 'getent group wheel' or 'lid -g wheel', then remove the user from the group with 'gpasswd -d username wheel' (this only strips group membership, it doesn't touch their home directory or files). If they're leaving the org entirely I'd lock the account first with 'usermod -L' or 'passwd -l', confirm no cron jobs or running processes belong to them, then decide separately whether to archive or delete their home directory rather than doing it in the same step as the group change.

```
getent group wheel
gpasswd -d username wheel
usermod -L username
```

Q4. What's the difference between a user's primary group and supplementary groups, and where does that distinction actually matter operationally?

Answer:

The primary group is recorded in /etc/passwd (GID field) and is the group assigned to any file the user creates by default; supplementary groups are listed in /etc/group and just grant additional access without becoming the file owner group. It matters operationally with services: for example an application running as a service account might need a supplementary group to read a shared log directory, but you wouldn't want that group to become the owner of every file the service writes — so I add it as supplementary with usermod -aG rather than changing the primary group with usermod -g.

```
usermod -aG logreaders appuser # supplementary, safe
# usermod -g logreaders appuser # changes primary group - usually NOT what you want
```

3. Permissions (Basic)

Q1. A web developer complains they can edit index.html locally but the deployed file on the server won't save via SFTP even though 'ls -l' shows rw for their user. What do you check?

Answer:

Standard rwx permissions on the file are necessary but not sufficient — I'd check three more layers: (1) directory permissions, since you need write+execute on the parent directory to create/rename a file even if the file itself is writable; (2) SELinux context with 'ls -Z', since a mismatched context (e.g. default_t instead of httpd_sys_content_t) can silently block writes that DAC permissions allow; (3) disk quota or read-only mount. In practice on RHEL, SELinux is the most common 'phantom permission denied' cause when the classic rwx bits look fine.

```
ls -ld /var/www/html
ls -Z /var/www/html/index.html
restorecon -Rv /var/www/html
```

Q2. Explain, as if to a manager, why 'chmod 777' on a shared upload directory is a bad idea even though it 'fixes' the permission error.

Answer:

777 gives read, write, and execute to the owner, group, and everyone else — meaning any local user or compromised process on the box can read, overwrite, or delete files in that directory, including files belonging to other users. It solves the symptom but opens a security hole. The correct fix is almost always to put the right user/group ownership in place and use the minimum permission that works — often 770 with proper group membership, or 2770 with SGID so new files inherit the right group — rather than opening it to the world.

```
chmod 770 /shared/uploads
chown appuser:appteam /shared/uploads
```

Q3. A cron job owned by a service account fails with 'permission denied' even though the script has execute permission for everyone. What else could be wrong?

Answer:

I'd check whether the script's shebang interpreter itself is executable and in the account's PATH, whether the parent directories are traversable (x bit) by that account all the way up the path, and whether the target file the script writes to is actually writable by that specific user — 'rwx for everyone' on the script doesn't help if it tries to write into a directory it can't access. I'd also check SELinux and, if it's a systemd timer instead of classic cron, the unit's User= and WorkingDirectory= settings.

```
sudo -u svc_account bash -x /path/script.sh # reproduce as that exact user
```

Q4. What is umask and how would you set a org-wide default so that new files created by any user are never world-writable?

Answer:

umask is the mask subtracted from the default permission (666 for files, 777 for directories) at creation time, so a umask of 022 gives files 644 and directories 755 by default. To enforce it system-wide I'd set it in /etc/profile or a script in /etc/profile.d/ rather than relying on each user's

personal shell rc file, so it applies consistently regardless of which shell or account is used.

```
echo 'umask 027' > /etc/profile.d/umask.sh
chmod +x /etc/profile.d/umask.sh
```

4. Advanced Permissions (ACLs)

Q1. A finance folder needs read access for the 'finance' group AND separately for one contractor account who isn't part of that group, without changing standard ownership. How?

Answer:

Standard Unix permissions only give you one user + one group, so I'd use POSIX ACLs to layer an additional entry for the contractor without disturbing existing ownership or the finance group's access. setfacl lets me grant that one account read-only access directly on the directory (and recursively on existing files), and I'd verify it afterward with getfacl.

```
setfacl -m u:contractor1:r-x /finance/reports
setfacl -R -m u:contractor1:r-x /finance/reports/*
getfacl /finance/reports
```

Q2. How do you make sure that ACL you just set on a directory also applies automatically to every new file created inside it later?

Answer:

A regular ACL entry only applies to what exists at the time you run setfacl. To make it persistent for future files, I set a default ACL (using the 'd:' prefix) on the directory — that acts like an inherited permission template, similar in spirit to setgid but far more granular since it can target specific individual users or groups.

```
setfacl -d -m u:contractor1:r-x /finance/reports
getfacl /finance/reports # shows both normal and default: entries
```

Q3. A user says 'ls -l' shows 'rwxr-xr-x' on a file but they're still getting extra restrictions/permissions they don't understand. What's the tell-tale sign an ACL is involved, and how do you inspect it?

Answer:

The tell-tale sign is a '+' at the end of the permission string in 'ls -l' output (e.g. -rwxr-xr-x+), which means there's an extended ACL beyond the classic owner/group/other bits. I'd run getfacl on the file to see the full list of user/group ACL entries and the effective mask, since the mask entry can silently cap what an ACL grants even if an individual entry looks more permissive.

```
ls -l file.txt
# -rw-r--r--+ 1 root root ... file.txt
getfacl file.txt
```

Q4. How do you remove all ACLs from a directory tree and go back to plain standard permissions?

Answer:

I use setfacl with -b to remove all extended ACL entries, or -R for recursive removal across a whole tree, then confirm with getfacl that only the base owner/group/other entries remain and the '+' is gone from ls -l.

```
setfacl -R -b /finance/reports
getfacl /finance/reports
ls -ld /finance/reports
```

5. Special Permissions (SUID / SGID / Sticky Bit)

Q1. You run 'ls -l /usr/bin/passwd' and see an 's' in the owner execute position. Explain what that means and why it's necessary.

Answer:

That 's' is the SUID bit. It means that when any regular user runs /usr/bin/passwd, the process temporarily executes with the privileges of the file's owner (root) instead of the invoking user. This is necessary because changing a password requires writing to /etc/shadow, which normal users can't touch directly — SUID lets a trusted, narrowly-scoped binary do that write on their behalf without granting the user root access generally.

```
ls -l /usr/bin/passwd
# -rwsr-xr-x. 1 root root ... /usr/bin/passwd
```

Q2. A security audit flags several unexpected SUID binaries on a server. How do you find them all and decide what to do?

Answer:

I'd sweep the filesystem for any file with the SUID or SGID bit set, cross-reference the list against what's expected for a stock RHEL install and installed packages (rpm -V can help spot ones that don't match package expectations), and remove the bit from anything that isn't explicitly required — since every SUID binary is a potential privilege-escalation vector if it has a bug.

```
find / -perm -4000 -type f 2>/dev/null # SUID
find / -perm -2000 -type f 2>/dev/null # SGID
chmod u-s /path/to/suspicious-binary
```

Q3. Why does /tmp have permissions like 'drwxrwxrwt' instead of something more restrictive, and what would happen if you removed that final 't'?

Answer:

/tmp needs to be writable by every user on the system so any process can create temp files there, but without the sticky bit any user could delete or rename another user's files in that shared space. The sticky bit ('t') restricts deletion/renaming inside the directory to the file's owner (or root), even though everyone can still write new files. Removing it would mean any local user could delete or overwrite files created by other users or services in /tmp — a real security and stability risk.

```
chmod +t /tmp
ls -ld /tmp
# drwxrwxrwt
```

Q4. How would you set up a shared team directory where (a) everyone in the group can create/edit files, (b) new files always belong to the team group, and (c) users can't delete each other's files?

Answer:

That's the classic combination of SGID plus the sticky bit on the same directory: SGID (2) makes new files inherit the directory's group automatically, and sticky (1) prevents one member from deleting another member's files even though the group has write access. Numerically that's mode 3770 (or 3775 if others need read/execute).

```
chown root:team /shared/team
chmod 3770 /shared/team
ls -ld /shared/team
# drwxrws--T
```

6. Package Management (RPM / DNF / YUM)

Q1. A deployment requires installing a specific older version of a package for compatibility, but dnf keeps pulling the latest. How do you pin it?

Answer:

I'd install the exact version using the name-version-release syntax, then use dnf's 'versionlock' plugin so future 'dnf update' runs skip that package instead of silently bumping it and breaking compatibility again.

```
dnf install package-1.2.3-1.el9.x86_64
dnf install python3-dnf-plugin-versionlock
dnf versionlock add package
dnf versionlock list
```

Q2. You copied an RPM file to a server with no internet access and 'rpm -i package.rpm' fails with unresolved dependencies. What do you do?

Answer:

rpm alone doesn't resolve dependencies from repos, so I'd either bring the dependency RPMs across as well and install them together in one rpm -Uvh call (so RPM can resolve the set), or better, set up a local/offline repo — mount the RHEL ISO or sync a repo with reposync from a machine that has connectivity — and register it so dnf can pull in dependencies automatically.

```
rpm -Uvh package.rpm dependency1.rpm dependency2.rpm
# OR create local repo:
mount /dev/sr0 /mnt/rhel9dvd
createrepo /mnt/rhel9dvd
# add .repo file pointing baseurl=file:///mnt/rhel9dvd
```

Q3. A production server has a package installed that appears corrupted — the binary crashes but you're not sure if files were tampered with or just missing. How do you verify package integrity?

Answer:

I'd use 'rpm -V' (verify) on the package, which compares installed files against the recorded checksums, permissions, size, and owner in the RPM database and flags any discrepancy with a coded output (e.g. '5' for MD5 mismatch, 'M' for mode). If it's genuinely corrupted I reinstall cleanly with 'dnf reinstall packagename' rather than trying to patch individual files.

```
rpm -V httpd
dnf reinstall httpd
```

Q4. Explain the practical difference between dnf and rpm to someone who's only used apt before.

Answer:

rpm is the low-level package tool — it installs, queries, and verifies individual .rpm files but has no concept of remote repositories or automatic dependency resolution. dnf (successor to yum) is the higher-level package manager that wraps rpm, talks to configured repositories in /etc/yum.repos.d/, resolves and pulls in dependencies automatically, and handles updates/removals cleanly — it's the RHEL equivalent of apt, while rpm is closer to dpkg.

```
rpm -qa | grep httpd # low-level query
dnf install httpd # high-level, resolves deps from repo
```

7. Repository Server / Repository Client

Q1. Your production servers have no direct internet access. How do you give them access to RHEL packages without connecting them to Red Hat's CDN directly?

Answer:

I'd stand up an internal repo server: sync the required repos onto it using reposync (or mount the RHEL DVD/ISO for a simple offline case), run createrepo to build the repodata, then serve it over HTTP with something like Nginx or Apache. Each client then gets a .repo file in /etc/yum.repos.d/ pointing baseurl at the internal server instead of Red Hat's CDN, so all package installs stay inside the network.

```
reposync --repoid=rhel-9-baseos -p /repo
createrepo /repo/rhel-9-baseos
# client:
cat >/etc/yum.repos.d/internal.repo <<EOF
[internal-baseos]
name=Internal BaseOS
baseurl=http://reposrv.local/rhel-9-baseos
enabled=1
gpgcheck=0
EOF
```

Q2. A client server runs 'dnf update' and gets 'Could not resolve host' or repo errors even though the repo server is reachable by ping. What do you check?

Answer:

Ping succeeding just confirms L3 connectivity, not that the repo service itself is healthy. I'd check that the web service (httpd/nginx) on the repo server is actually running and listening on the expected port, that the firewall on the repo server allows that port, that the baseurl in the client's .repo file is correct and resolvable via DNS, and I'd manually curl the repodata URL from the client to isolate whether it's a DNS, firewall, or web-server-config issue.

```
curl -I http://reposrv.local/rhel-9-baseos/repodata/repomd.xml
systemctl status httpd
firewall-cmd --list-all
```

Q3. After adding new RPMs to your local repo directory, clients still don't see them as available packages. Why, and how do you fix it?

Answer:

The repodata (the metadata index dnf actually reads) isn't automatically regenerated when you drop new files into the directory — you have to re-run createrepo (with --update for speed) to refresh repodata/repomd.xml, and then clients need to clear their local dnf cache so they don't keep serving the stale metadata they already downloaded.

```
createrepo --update /repo/rhel-9-baseos
# on client:
dnf clean all
dnf makecache
```

Q4. What's the purpose of GPG signing/checking in a repo setup, and when is it reasonable to disable gpgcheck?

Answer:

GPG checking verifies that packages haven't been tampered with by validating their signature against a trusted key, which matters a lot for anything coming from the public internet. For a fully internal, air-gapped repo that I built myself from a trusted source (like a mounted vendor ISO) inside a controlled network, disabling gpgcheck is a common pragmatic shortcut in a lab/internal setting, but for anything customer- or internet-facing I'd keep gpgcheck=1 and import the correct signing key instead of turning verification off.

```
rpm --import /path/to/RPM-GPG-KEY-redhat-release
# repo file: gpgcheck=1
# gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

8. Networking

Q1. A newly deployed VM can't reach the internet or other hosts even though 'ip a' shows it has an IP address. Walk through your troubleshooting order.

Answer:

I work outward in layers: first confirm the interface is up and has the expected IP/subnet with 'ip a' and that the link state is UP; then check the default route with 'ip r'; then test whether it's a routing vs DNS problem by pinging a raw IP like 8.8.8.8 versus pinging a hostname; then check /etc/resolv.conf or the NetworkManager connection's DNS setting if raw IP works but names don't; and finally check firewalld/SELinux and the hypervisor's virtual switch/port group if everything on the guest looks correct but traffic still doesn't leave.

```
ip a
ip r
ping -c3 8.8.8.8
ping -c3 google.com
nmcli con show
cat /etc/resolv.conf
```

Q2. You need to configure a static IP on a RHEL 9 server managed by NetworkManager without breaking connectivity mid-change over SSH. How do you do it safely?

Answer:

I use nmcli to modify the connection profile's IPv4 settings (address, gateway, DNS) and set the method to manual, but I only apply/restart the connection at the very end — and ideally I do it from console/iDRAC/vSphere console rather than the same SSH session I'm editing over, in case the new config is wrong and drops my session. nmcli connection modify stages the change; 'nmcli con up' is the point where it actually takes effect.

```
nmcli con show
nmcli con mod ens192 ipv4.addresses 192.168.1.50/24
nmcli con mod ens192 ipv4.gateway 192.168.1.1
nmcli con mod ens192 ipv4.dns 192.168.1.1
nmcli con mod ens192 ipv4.method manual
nmcli con up ens192
```

Q3. Two VMs on the same subnet can't talk to each other, but both can reach the gateway fine. What's your investigation path?

Answer:

Since both can reach the gateway, basic addressing and gateway routing are fine, so I'd suspect something blocking direct host-to-host traffic: firewalld on either VM blocking the specific port/zone, a hypervisor-level isolation setting (like an ESXi port group with a security policy or a Tailscale/VPN ACL if they're on an overlay network), or an ARP/switch issue if they're technically on different VLANs despite appearing to share a subnet. I'd use tcpdump on both ends simultaneously to see whether the packet even arrives, which tells me if it's a network path issue or a host firewall issue.

```
tcpdump -i any host <other-vm-ip>
firewall-cmd --list-all
ip route get <other-vm-ip>
```

Q4. What's the difference between nmcli, nmtui, and directly editing files in /etc/NetworkManager, and when would you use each?

Answer:

nmcli is the scriptable command-line tool — best for automation, Ansible tasks, or quick precise changes. nmtui is a text UI wrapper around the same NetworkManager backend, handy for a quick interactive fix from a console when you don't remember exact nmcli syntax. Directly editing the keyfiles under /etc/NetworkManager/system-connections/ is lower-level and useful for templating configs or troubleshooting, but manual edits require a 'nmcli con reload' to take effect and are more error-prone since NetworkManager expects specific formatting.

```
nmcli # scripting/automation
nmtui # interactive console
# /etc/NetworkManager/system-connections/*.nmconnection -- low-level
```

9. Firewall (firewalld)

Q1. An application team asks you to open port 8443 for a new service, but only from the internal 10.0.0.0/8 network, not from everywhere. How do you configure that?

Answer:

A plain 'firewall-cmd --add-port' would open it to the whole zone, which is too broad. Instead I'd create a rich rule scoped to that specific source network, or place the interface in a zone dedicated to internal traffic and add the port rule to that zone only, then make it permanent and reload so it survives a restart.

```
firewall-cmd --permanent --zone=internal \
--add-rich-rule='rule family="ipv4" source address="10.0.0.0/8" port port="8443"
protocol="tcp" accept'
firewall-cmd --reload
firewall-cmd --list-all --zone=internal
```

Q2. A colleague ran 'firewall-cmd --add-port=8080/tcp' and it worked, but after a server reboot the port was closed again. What went wrong and how do you prevent it?

Answer:

firewall-cmd changes made without --permanent only affect the live runtime configuration, not the saved configuration, so they're lost on reload/reboot. The fix is to always pair a runtime change with the permanent flag (or add it directly as permanent and reload), and it's good practice to add both the runtime and permanent rule together during a live troubleshooting session so the fix is immediate AND persists.

```
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
firewall-cmd --list-ports
```

Q3. A web server won't accept connections on port 443 even though 'firewall-cmd --list-all' shows https is allowed. What else could be blocking it at the firewall layer?

Answer:

I'd double check I'm looking at the zone the interface is actually bound to (firewalld can have multiple zones and the active interface might not be in the zone I edited), then check for masquerade/NAT issues if this is a routed/forwarded scenario, and check whether the service is actually listening on that port at all with ss -tlnp — since a closed firewall port and a service that isn't listening produce a similar symptom from the client's point of view. I'd also confirm SELinux isn't blocking the port with semanage port -l, since firewalld and SELinux are two independent layers that both must allow traffic.

```
firewall-cmd --get-active-zones
ss -tlnp | grep 443
semanage port -l | grep http_port_t
```

Q4. Explain firewalld zones to someone unfamiliar with them — why not just have one global set of allowed ports?

Answer:

Zones let you apply different trust levels to different network interfaces on the same box. For example a server might have one NIC facing the internal management network (zone: internal, more permissive) and another facing a DMZ or public network (zone: public, much more restrictive). Having a single global rule set would force you to choose between too permissive or too restrictive for one of those interfaces; zones let each interface get exactly the policy appropriate to what's connected to it.

```
firewall-cmd --get-zones
firewall-cmd --get-active-zones
nmcli con mod ens224 connection.zone internal
```

10. SELinux

Q1. Apache is configured correctly, the port is open in firewalld, but users still get 403 Forbidden when the document root is on a non-standard path like /data/www. What's the likely cause and fix?

Answer:

This is a textbook SELinux context mismatch. Files outside the default /var/www/html get labeled with a generic context like default_t instead of httpd_sys_content_t, and SELinux blocks httpd from serving them even though standard Unix permissions are fine. I'd confirm with 'ls -Z', check /var/log/audit/audit.log for AVC denials, then fix it properly by adding a persistent file-context rule with semanage (not just chcon, which doesn't survive a relabel) and applying it with restorecon.

```
ls -Z /data/www
grep AVC /var/log/audit/audit.log | tail
semanage fcontext -a -t httpd_sys_content_t "/data/www(/.*)?"
restorecon -Rv /data/www
```

Q2. A developer asks you to just 'turn off SELinux' because it's blocking their app and they're in a hurry. How do you respond, and what do you actually do instead?

Answer:

I'd push back on disabling it outright — SELinux is a major defense-in-depth layer and a permanent 'setenforce 0' or disabling it in /etc/selinux/config removes that protection system-wide, not just for this app. Instead I'd diagnose the actual denial with ausearch/audit2why, and either apply the correct semanage fcontext label, open the specific SELinux boolean if one exists for this exact behavior (e.g. httpd_can_network_connect), or generate a narrowly-scoped custom policy module with audit2allow if it's a genuinely unusual requirement — fixing the real cause instead of switching SELinux off.

```
ausearch -m avc -ts recent | audit2why
getsebool -a | grep httpd
setsebool -P httpd_can_network_connect on
```

Q3. You need a web app to connect out to a custom database port (say 9999/tcp) that SELinux doesn't know is a database port. How do you allow that without disabling enforcement?

Answer:

I'd add that port to the correct SELinux port type using semanage port, so SELinux treats connections to 9999 as an allowed database-type connection for httpd, rather than loosening enforcement generally.

```
semanage port -a -t http_port_t -p tcp 9999
semanage port -l | grep 9999
```

Q4. What's the difference between Enforcing, Permissive, and Disabled SELinux modes, and when (if ever) would you legitimately use Permissive in production?

Answer:

Enforcing actively blocks and logs anything that violates policy; Permissive logs what WOULD have been blocked but allows it anyway; Disabled turns SELinux off entirely and stops even logging. I'd

use Permissive temporarily and deliberately as a diagnostic tool — for example right after deploying a new app, to capture exactly which AVC denials it triggers over a day of real traffic, so I can build the precise semanage/booleans/policy fixes needed — then switch back to Enforcing once those fixes are applied. I wouldn't leave a production system in Permissive long-term since that's effectively 'no protection, but I'll know about it.'

```
getenforce
setenforce 0 # Permissive, temporary, for diagnosis
setenforce 1 # back to Enforcing
```

11. Satellite (Foreman/Katello)

Q1. Your organization wants centralized patch management and content lifecycle across 200 RHEL servers instead of each admin running dnf update independently. How does Satellite/Foreman-Katello solve this?

Answer:

Satellite (built on Foreman + Katello upstream, which I've deployed hands-on via foremanctl) acts as the single source of truth for content: it syncs Red Hat repos once, organizes packages/errata into Content Views that are versioned and promoted through Lifecycle Environments (Dev -> Test -> Prod), and registers all 200 clients against it instead of Red Hat's CDN directly. That means every environment gets a known, tested, and reproducible set of packages, patching is done in controlled waves by promoting a Content View version rather than an uncoordinated 'dnf update' on every box, and you get a single dashboard for errata/compliance across the whole fleet.

Q2. Setting up my own Foreman/Katello deployment, I hit cascading errors — Ruby version conflicts, then PostgreSQL corruption, then a Redis port collision. How would you approach debugging a multi-layered failure like that?

Answer:

I resolve dependency chains from the bottom up rather than chasing the top-level error first, since surface errors are often just symptoms cascading from an earlier failed component. In that case I first pinned the correct Ruby version the installer expected (rbenv/scl) to get past the Ruby conflict, then addressed the PostgreSQL corruption directly — stopping the service, checking data directory integrity, and in my case reinitializing the DB cleanly rather than patching a corrupted cluster — and only once the DB was healthy did I look at the Redis port collision, which turned out to be a leftover process from a previous failed install attempt still bound to the port. Checking service status and logs after fixing each layer, instead of applying all fixes at once, made it possible to actually tell which fix resolved which symptom.

```
systemctl status foreman postgresql redis
journalctl -u foreman -n 100
ss -tlnp | grep 6379 # find what's holding the Redis port
```

Q3. A client host registered to Satellite isn't showing up with current errata/package data. What do you check?

Answer:

I'd verify the katello-agent/host registration is intact and the host's subscription-manager status is current, check that the correct Content View and Lifecycle Environment are assigned to that host, confirm the host can actually reach the Satellite server's Pulp content endpoints over the network/firewall, and manually trigger a package profile update rather than waiting for the next scheduled check-in.

```
subscription-manager status
subscription-manager repos --list
dnf clean all && dnf makecache
# on Satellite: check host details -> Content -> Errata
```

Q4. How does Satellite's Content View / Lifecycle Environment model help prevent an untested patch from breaking production?

Answer:

Content Views snapshot a specific, versioned set of repositories and errata rather than always tracking 'latest'. You promote a Content View version through Lifecycle Environments in order — say Library -> Dev -> QA -> Production — testing at each stage before promoting further. Production hosts are only ever subscribed to whatever Content View version is currently assigned to the Production environment, so a bad patch that's still sitting in Dev or QA simply can't reach production servers until someone deliberately promotes it forward.

12. Web Server (Apache/Nginx)

Q1. You deployed a three-tier app (Nginx + Flask + MySQL) on RHEL and Nginx returns 502 Bad Gateway. How do you isolate whether the problem is Nginx, the app, or something in between?

Answer:

502 specifically means Nginx received a bad/no response from the upstream, so I start at the app layer: is the Flask app (likely behind gunicorn/uwsgi) actually running and listening on the socket/port Nginx is configured to proxy to? I check with ss -tlnp or by curling the app directly on localhost, bypassing Nginx. If the app responds fine locally, the issue is in the Nginx proxy_pass config, a permissions issue on a Unix socket, or SELinux blocking Nginx from making that network connection (httpd_can_network_connect boolean). If the app itself isn't listening, then it's an app/service issue, not Nginx.

```
curl -v http://127.0.0.1:8000/ # hit the app directly, bypass nginx
ss -tlnp | grep 8000
tail -f /var/log/nginx/error.log
getsebool httpd_can_network_connect
```

Q2. A site works over HTTP but fails to load over HTTPS with a certificate error after you added a TLS cert. What's your checklist?

Answer:

I'd verify the cert and key actually match and aren't expired (openssl x509 -noout -dates / -modulus comparison), confirm the Nginx/Apache config points to the correct cert chain including any intermediate certificates (a missing intermediate is the single most common cause of browser trust errors even when the cert itself is valid), check that port 443 is open in firewalld and the httpd/nginx SELinux context allows it, and finally confirm the server is actually reloaded (not just the config file edited) after the cert was added.

```
openssl x509 -in cert.pem -noout -dates
openssl x509 -noout -modulus -in cert.pem | md5sum
openssl rsa -noout -modulus -in key.pem | md5sum
nginx -t && systemctl reload nginx
```

Q3. How would you configure Nginx as a reverse proxy load-balancing traffic across three backend app instances, and what health-check consideration matters?

Answer:

I'd define an upstream block listing the three backend addresses, then proxy_pass the server block to that upstream name, choosing a balancing method (round-robin by default, or least_conn if requests vary a lot in duration). Critically, I'd also configure passive health checks (max_fails/fail_timeout) or use a health-check module so Nginx automatically stops sending traffic to a backend that's returning errors or timing out, instead of continuing to round-robin into a dead instance.

```
upstream app_backend {
least_conn;
server 10.0.0.11:8000 max_fails=3 fail_timeout=30s;
server 10.0.0.12:8000 max_fails=3 fail_timeout=30s;
server 10.0.0.13:8000 max_fails=3 fail_timeout=30s;
```

```
}
server {
listen 80;
location / { proxy_pass http://app_backend; }
}
```

Q4. Your Apache-based site serves static content fine, but a subdirectory with uploaded images returns 403 even though file permissions look correct. What do you check specific to Apache/RHEL?

Answer:

Beyond standard file permissions, on RHEL this is almost always SELinux context (httpd_sys_content_t vs whatever the upload process actually set the new files to) or an explicit Apache config directive — a <Directory> block with 'Require all denied' left over from a template, or an .htaccess restriction if AllowOverride is enabled for that path. I check the httpd error log first since it usually names the exact reason (SELinux denial vs config-level deny).

```
tail -f /var/log/httpd/error_log
ls -Z /var/www/html/uploads
apachectl -S # dump config, check Directory blocks
```

13. Migration

Q1. You need to migrate an application from an on-prem RHEL 8 server to a new RHEL 9 VM with zero data loss and minimal downtime. What's your plan?

Answer:

I break it into pre-migration, cutover, and validation phases. Pre-migration: stand up the new RHEL 9 target, replicate application data ahead of time with rsync (running it multiple times to shrink the delta), and mirror package/config state using the same Ansible playbooks that built the original box, so the target is a known-good, already-tested replica before cutover night. Cutover: put the app in maintenance mode, run one final rsync delta to catch last-minute changes, switch DNS/load-balancer target (or IP) to the new box. Validation: smoke-test the app end-to-end, watch logs for errors, and only decommission the old server after a defined stable-observation window, keeping it as a rollback option in the meantime.

```
rsync -avz --delete /app/data/ newserver:/app/data/ # run pre-cutover, then
final delta at cutover
# keep old server powered off (not deleted) for N days as rollback
```

Q2. Migrating a database-backed app, how do you decide between a big-bang cutover versus a phased/parallel-run migration?

Answer:

It comes down to acceptable downtime, data-consistency requirements, and rollback risk tolerance. Big-bang (stop old, migrate, start new) is simpler to reason about and appropriate for lower-traffic or internal systems where a maintenance window is acceptable. A phased/parallel-run approach — running old and new in parallel with data replication or dual-writes, gradually shifting read/write traffic — minimizes downtime and gives you a live fallback, but it's operationally more complex and needs careful handling of data consistency during the overlap window. For anything customer-facing with strict uptime SLAs I'd lean toward phased; for an internal tool, big-bang during a scheduled window is often the pragmatic choice.

Q3. After migrating a VM from one hypervisor to another (or across ESXi hosts), the guest boots but networking is broken — the interface name changed or the IP didn't come up. Why does this happen and how do you fix it?

Answer:

This usually happens because the new virtual NIC gets a different device name (udev/NetworkManager ties connection profiles to specific MAC addresses or interface names) and the old profile doesn't match the newly detected hardware, so the interface stays unconfigured. I'd check 'nmcli con show' and 'ip a' to see the actual current interface name, then either update the existing connection profile's interface-name/MAC binding or create a fresh nmcli connection matching the current NIC and bring it up.

```
ip a # find actual current interface name
nmcli con show # see what NetworkManager thinks exists
nmcli con mod "old-profile" connection.interface-name ens192
nmcli con up "old-profile"
```

Q4. You're migrating configuration management from ad-hoc manual server setup to Ansible. How do you approach converting an existing fleet without breaking anything that's already running?

Answer:

I don't try to convert everything at once. I start by writing playbooks/roles that describe the CURRENT state (not an idealized one) and run them in check mode (--check --diff) against existing servers first, to see exactly what Ansible thinks it would change without actually changing anything. Any unexpected diff tells me the playbook doesn't yet accurately reflect reality, and I fix the playbook, not the server. Only once check-mode runs show zero unexpected drift do I run it for real, starting with one non-critical server, then rolling out to the rest of the fleet in batches using --limit, so a bad assumption in the playbook is caught early and contained.

```
ansible-playbook site.yml --check --diff --limit test-server
ansible-playbook site.yml --limit test-server
ansible-playbook site.yml --limit "batch1"
```

14. Monitoring through Prometheus

Q1. You need to monitor CPU, memory, and disk usage across your homelab VMs (Ansible control node, DNS VM, Foreman node) in one place. How do you set this up with Prometheus?

Answer:

I'd install node_exporter on each VM to expose host-level metrics (CPU, memory, disk, network) on port 9100, then configure a central Prometheus server with a scrape_configs job listing each node as a target so Prometheus pulls metrics on an interval. On top of that I'd run Grafana pointed at Prometheus as its data source to build dashboards, since Prometheus's own UI is good for ad-hoc PromQL queries but Grafana is what you actually want for a persistent visual dashboard.

```
# prometheus.yml
scrape_configs:
- job_name: 'homelab_nodes'
static_configs:
- targets: ['ansible-ctrl:9100', 'dns-vm:9100', 'foreman:9100']

systemctl enable --now node_exporter
curl http://ansible-ctrl:9100/metrics | head
```

Q2. A target in Prometheus shows as 'DOWN' in the Targets page. What's your troubleshooting order?

Answer:

First I check basic reachability from the Prometheus server to the exporter's port with curl/telnet, since a DOWN target most often means the scrape itself failed. If that fails, I check whether node_exporter (or whichever exporter) is actually running on the target and listening on the expected port, then check firewalld on the target for that port, and finally check the Prometheus server logs for the specific scrape error (timeout vs connection refused vs TLS error), since each points to a different layer of the problem.

```
curl http://target-host:9100/metrics
systemctl status node_exporter
firewall-cmd --list-ports
journalctl -u prometheus -n 50
```

Q3. How would you set up an alert that pages you when disk usage on any server exceeds 90%, and why use Alertmanager instead of just eyeballing dashboards?

Answer:

Dashboards are for human-driven investigation; they don't page anyone at 2am. I'd write a PromQL alerting rule in Prometheus that evaluates disk usage percentage against a threshold, and route firing alerts to Alertmanager, which handles deduplication, grouping, silencing, and routing to a notification channel (email/Slack/PagerDuty). This separates 'detecting the condition' (Prometheus rule) from 'deciding who gets notified and how' (Alertmanager), which is important once you have more than a couple of alerts, so you don't get duplicate or noisy pages.

```
# alert.rules.yml
groups:
- name: disk_alerts
```

```
rules:
- alert: DiskSpaceCritical
expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 10
for: 5m
labels:
severity: critical
annotations:
summary: "Disk usage above 90% on {{ $labels.instance }}"
```

Q4. What's the difference between Prometheus's pull model and a push-based monitoring approach, and why does Prometheus default to pull?

Answer:

In Prometheus's pull model, the server actively scrapes each configured target's /metrics endpoint on a schedule; in a push model, agents on each host push metrics to a central collector. Pull makes it trivial for Prometheus to know a target's health simply by whether the scrape succeeds (a failed scrape IS the 'target down' signal), centralizes scrape-interval and target configuration in one place, and avoids agents needing to know where to push or handling backpressure. It's a weaker fit for very short-lived jobs (like a batch job that finishes before a scrape interval), which is why Prometheus provides the separate Pushgateway component specifically for that case rather than changing the core model.

15. Linux Upgrade: RHEL 9 to RHEL 10

Q1. Your organization wants to upgrade a fleet of RHEL 9 servers to RHEL 10. What's your high-level upgrade strategy, and why not just run dnf upgrade across major versions?

Answer:

A major version upgrade (9 to 10) isn't a simple 'dnf upgrade' the way a minor point release is — it involves new default package sets, potential removal/replacement of deprecated components, and toolchain changes, so Red Hat provides a dedicated in-place upgrade tool (Leapp) for supported major-version jumps. My strategy: (1) inventory the fleet and check Red Hat's official upgrade paths/compatibility matrix for RHEL 9 -> 10, (2) take a full backup/snapshot of each system before touching it, (3) run the Leapp pre-upgrade check in report-only mode first to surface blockers (unsupported packages, custom kernel modules, unsupported repos) without changing anything, (4) resolve every blocker the report identifies, (5) pilot the actual upgrade on a small non-critical batch first, and only then roll out to the rest of the fleet in controlled waves.

```
leapp preupgrade # generates report, changes nothing
cat /var/log/leapp/leapp-report.txt
# resolve every 'inhibitor' before proceeding
leapp upgrade # actual upgrade, after reboot into upgrade env
```

Q2. The Leapp pre-upgrade report flags an 'inhibitor' related to a third-party kernel module and a deprecated package. How do you handle inhibitors versus warnings?

Answer:

Inhibitors are hard blockers — Leapp will refuse to proceed with the actual upgrade until they're resolved, because they represent something that would likely break the system if the upgrade continued (e.g., a kernel module with no RHEL 10-compatible build). Warnings are things worth reviewing but that won't stop the upgrade. For the kernel module, I'd check with the vendor for a RHEL 10-compatible version, or if it's not critical, remove/disable it before upgrading and reinstall an updated version post-upgrade. For the deprecated package, I'd identify its RHEL 10 replacement (Red Hat's release notes typically list these) and migrate the config/functionality to the replacement before attempting the upgrade again.

```
leapp preupgrade
# review /var/log/leapp/leapp-report.txt for 'inhibitor' entries specifically
# resolve each, then re-run leapp preupgrade to confirm a clean report
```

Q3. After a successful Leapp upgrade to RHEL 10, a custom application fails to start with library/dependency errors. How do you approach this?

Answer:

A major OS upgrade often bumps core library versions (glibc, openssl, python, etc.), so I'd first check the app's actual error for which library/symbol is missing, then check whether the app was compiled against or explicitly depends on an older library version rather than assuming it's a generic 'something's broken' issue. Options from there: install any compat packages Red Hat provides for that exact library version, rebuild/recompile the app against the new RHEL 10 toolchain, or containerize the app so its dependency versions are decoupled from the host OS going forward, which is often the more durable long-term fix for this exact class of problem.

```
ldd /path/to/app_binary # find missing/mismatched shared libs
rpm -q --whatprovides libssl.so.1.1
dnf search compat-openssl
```

Q4. How would you validate a RHEL 9 to RHEL 10 upgrade was fully successful before signing off, beyond just 'the server booted'?

Answer:

Booting is necessary but nowhere near sufficient. I'd confirm the running kernel and os-release actually report RHEL 10, verify all critical services are active and enabled (not just running by accident), re-check SELinux is enforcing with no new unexpected AVC denials appearing post-upgrade, confirm firewalld rules survived the upgrade intact, re-validate the app's full functional smoke test (not just 'process is up'), and check dnf/subscription-manager repo registration is correctly pointing at RHEL 10 repos rather than stale RHEL 9 entries.

```
cat /etc/os-release
uname -r
systemctl --failed
getenforce
ausearch -m avc -ts today
firewall-cmd --list-all
subscription-manager repos --list-enabled
```